

Data Structures and Algorithms

Doubly Linked List, Circularly Linked List



Lecture 4

Software Engineering Dep.

Second Stage

Dr.Saman M. Omer

2025-2026

Outline

2

- An Overview of Singly Linked List
- Doubly Linked List
- Circularly Singly linked list
- Circularly Doubly linked list
- Advantages and Disadvantages of Doubly and Circularly Linked List
- Basic Operations
- Implementing a Doubly and Circularly Linked List Class
- Inserting and Deleting a node in Doubly and Circularly Linked List
- Implementation

Types of Linked List

6

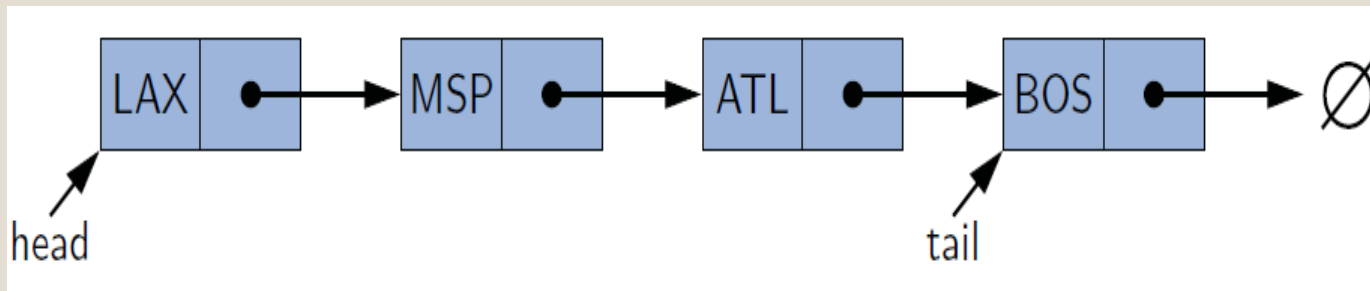
❑ Following are the various types of linked list.

- ❖ **Simple Linked List:** Item navigation is forward only.
- ❖ **Circular Linked List:** Circular Linked List is a variation of Linked list in which the first element points to the last element and the last element points to the first element.
- ❖ **Doubly Linked List:** is a variation of Linked list in which Items navigation is possible in both ways, either forward and backward

Singly Linked List

4

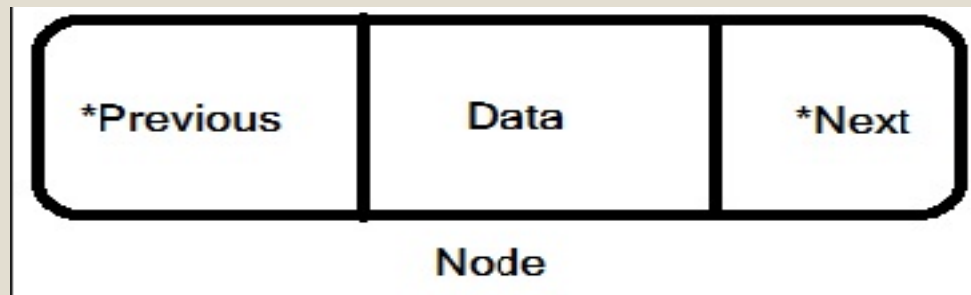
- ❑ Linked lists are traditionally viewed as storing a sequence of items in a linear order, from first to last.
- ❑ In a singly linked list, each node maintains a reference to the node that is immediately after it and the last node points to NULL.
- ❑ In a singly linked list we can not go to any the preceding nodes
- ❑ However, there are many applications in which data can be more naturally viewed as having a cyclic order, with well-defined neighboring relationships, but no fixed beginning or end.



Doubly Linked List

5

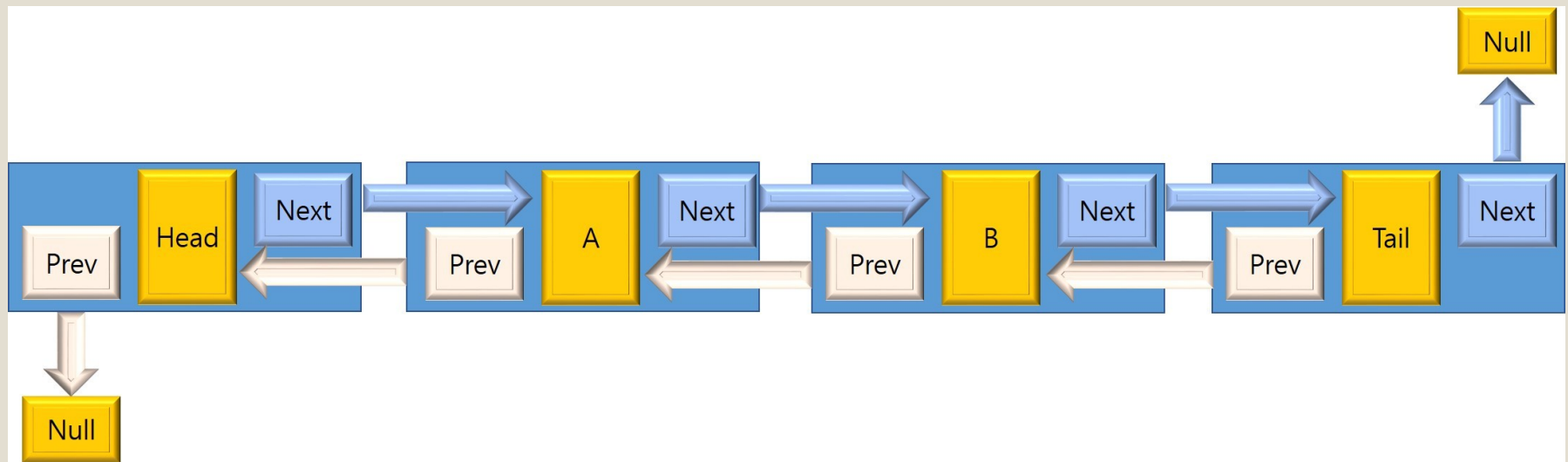
- ❑ **Double linked list** is a sequence of elements in which every element has links to its previous element and next element in the sequence.
- ❑ Doubly Linked List is a variation of Linked list in which navigation is possible in both ways, either forward and backward easily as compared to Single Linked List.
- ❑ Every node in a doubly linked list contains three fields:
 - **Data** : field is used to store the actual value of that node called an element.
 - **Next** : field is used to store the address of the **next** node in the sequence.
 - **Prev** : field is used to store the address of the **previous** node in the sequence.



Doubly Linked List Representation

6

- ❑ Always the **previous** field of the **first node** must be **NULL**.
- ❑ Always the **next** field of the **last node** must be **NULL**.



Advantages of a DLL over a SLL

Feature	Singly Linked List (SLL)	Doubly Linked List (DLL)	Advantage of DLL
Traversal Direction	Forward only (head → tail)	Both forward & backward (next & prev)	Bidirectional movement (useful for undo/redo, browser history)
Deletion of a Given Node	Requires previous node ($O(n)$ time in worst case)	Direct access via <code>prev</code> ($O(1)$ time if node is given)	Faster deletions without traversal
Insertion Before a Node	Must traverse from head ($O(n)$ time)	Direct access via <code>prev</code> ($O(1)$ time)	Efficient insertions before any node
Reverse Traversal	Requires reversing list ($O(n)$ time) or extra space	Possible in $O(n)$ time using <code>prev</code> pointers	No extra work needed for reverse traversal
Memory Overhead	Lower (only <code>next</code> pointer)	Higher (both <code>next</code> and <code>prev</code> pointers)	Trade-off: More memory for flexibility
Ease of Operations	Requires tracking previous node manually	Previous node always accessible via <code>prev</code>	Simplifies complex operations (e.g., swaps, merges)

Advantages and Disadvantages of Doubly Linked List

8

❑ **Advantages** over singly linked list :

- A DLL can be traversed in both forward and backward direction.
- The insert and delete operations in DLL is more efficient if pointer to the node to be deleted is given. We can get the previous node using previous pointer.

❑ **Disadvantages** over singly linked list :

- Every node of DLL Require extra space for an previous pointer.
- All operations require an extra pointer previous to be maintained. For example, in insertion, we need to modify previous pointers together with next pointers. we need 1 or 2 extra steps to set previous pointer.

When to Use DLL Over SLL?

9

□ Use DLL when:

- Backward traversal is needed (e.g., browser history).
- Frequent deletions/insertions at arbitrary positions.
- Reverse traversal is required without extra computation.

□ Use SLL when:

- Memory efficiency is critical.
- Only forward traversal is needed.
- Operations are mostly at the head/tail (like in stacks/queues).

- This summarizes why **DLL is more flexible but consumes more memory** than SLL.

Basic Operations

10

- ❑ Following are the basic operations supported by a Doubly Linked List.
 - **Insertion:** Adds a new Node at the beginning and at the end of the list.
 - **Deletion :** Deletes a node at the beginning and at the end of the list.
 - **Insert After :** Adds a new node after an item of the list.
 - **Delete :** Deletes a Node from the list using the key.
 - **Display forward :** Displays the complete list in a forward manner.
 - **Display backward :** Displays the complete list in a backward manner.
 - **Search:** Searches an element using the given key.

Implementing a Doubly Linked List Class

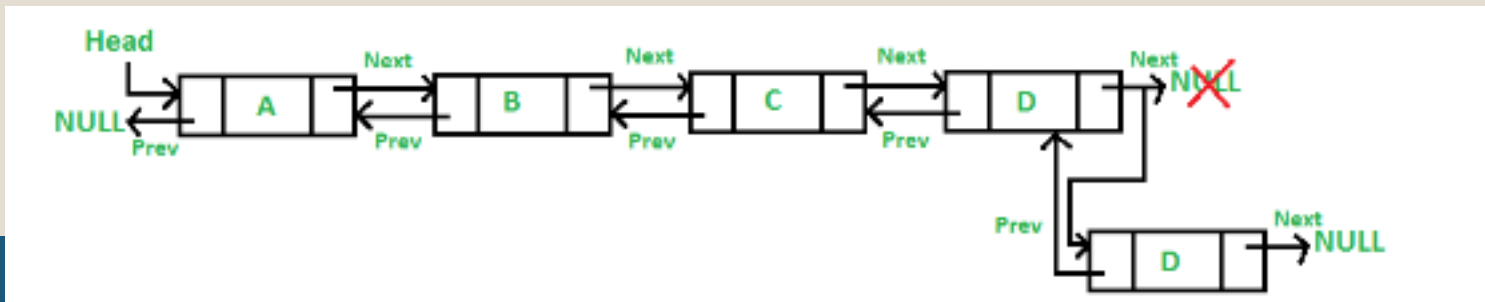
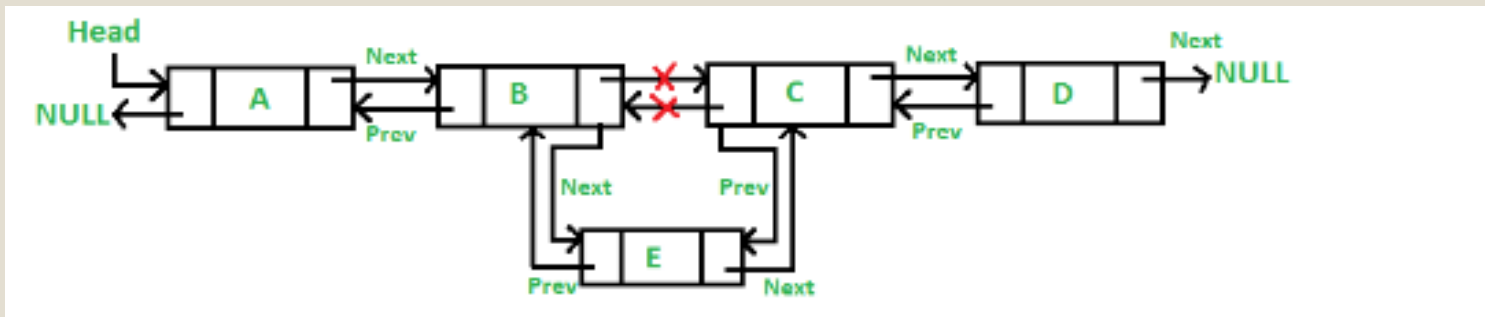
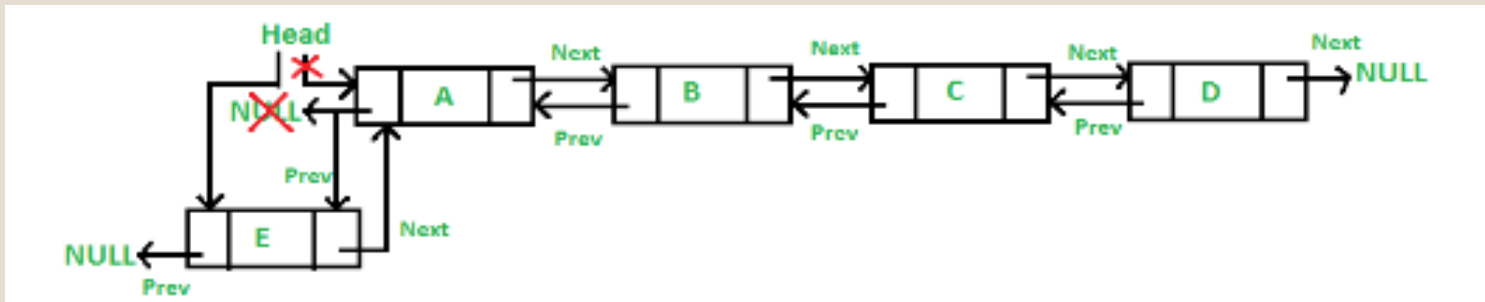
11

- ❖ **size()**: Returns the number of elements in the list.
- ❖ **isEmpty()**: Returns true if the list is empty, and false otherwise.
- ❖ **first()**: Returns (but does not remove) the first element in the list.
- ❖ **last()**: Returns (but does not remove) the last element in the list.
- ❖ **addFirst(e)**: Adds a new element to the front of the list.
- ❖ **addLast(e)**: Adds a new element to the end of the list.
- ❖ **removeFirst()**: Removes and returns the first element of the list.
- ❖ **removeLast()**: Removes and returns the last element of the list

Inserting a new node in Doubly Linked List

12

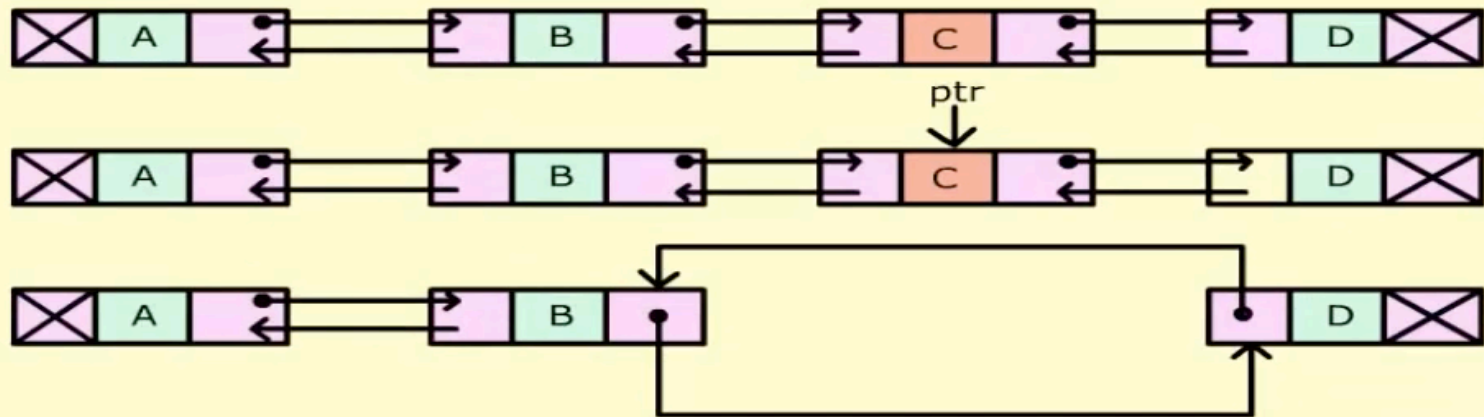
- ❑ Every insertion into a doubly linked list will take a place.
- ❑ Adding an element to the front, end and middle of a sequence doubly linked list



Deleting a node in Doubly Linked List

13

- ❑ The deletion of a node, proceeds in the opposite fashion of an insertion.
- ❑ The two neighbors of the node to be deleted are linked directly to each other
- ❑ As a result, that node will no longer be considered part of the list.



The Node implementation

14

```
public class DoublyLinkedList1<E> { // is a generic class that can hold
                                     elements of any type E
//----- nested Node class -----
    private static class Node<E> //represents individual nodes in the list
    { private E element; // Stores the actual data of type E
      private Node<E> next; // Reference to the next node in the list
      private Node<E> prev; //Reference to the previous node in the list
      public Node(E e, Node<E> p, Node<E> n)
          {element=e; prev=p; next=n;} // Node Constructor
      public E getElement(){return element;} // Returns the stored data
      public Node<E> getNext() {return next;}// Returns reference to
                                              next node
      public Node<E> getPrev() {return prev;}// Returns reference to
                                              previous node
      public void setNext(Node<E> n) {next=n;}// Updates the reference
                                              to next node
      public void setPrev(Node<E> p) {prev=p;}// Updates the reference
                                              to previous node
    } //----- end of nested Node class -----
```

Double Linked List constructors and methods

15

```
private Node<E> head=null;
private Node<E> tail=null;
private int size=0;
public DoublyLinkedList1(){};
public int size(){return size;}
public boolean isEmpty(){return size==0;}
```

```
public E first()
{if(isEmpty()) return null;
return head.getElement();
}
public E last()
{if(isEmpty()) return null;
return tail.getElement();
}
```

•
•
•

Double Linked List methods

16

```
public void addFirst(E e) {  
    Node<E> newest = new Node<E>(e, null, head);  
    if (head != null) {head.setPrev(newest);}  
    head=newest;  
    if (tail == null)  
        tail = newest;  
    size++;  
}
```

```
public void addLast(E e) {  
    Node<E> newest = new Node<E>(e, tail, null);  
    if (tail != null) {tail.setNext(newest);}  
    tail=newest;  
    if (head == null)  
        head = newest;  
    size++;  
}
```


Double Linked List methods

17

```
public E removeFirst( ) {  
    if (isEmpty( )) return null;  
    E removed = head.getElement();  
    head = head.getNext();  
    if (head!=null)  
        head.setPrev(null);  
    size--;  
    return removed;  
}
```

```
public E removeLast( ) {  
    if (isEmpty( )) return null;  
    E removed = tail.getElement( );  
    tail=tail.getPrev();  
    if(tail!=null)  
        tail.setNext(null);  
    size--;  
    return removed;  
}
```

Double Linked List methods

18

```
// display from first to last node
public void display()
{
    Node<E> current = head;
    for(int i=0;i<size ; i++) {
        System.out.print("{ " +current.getElement()+ " }");
        System.out.print("\t" );
        current = current.getNext();
    }
    System.out.print("\n" );
}
```

Practical operation

19

```
public static void main(String[] args)
{
    DoublyLinkedList1 dl=new DoublyLinkedList1();
    dl.addFirst(1);dl.addFirst(0);dl.addLast(2);dl.addLast(3);

    System.out.print(dl.removeFirst()+" "+dl.removeLast()+" "+
        dl.size()+"\n");
    System.out.print(dl.removeLast()+" "+dl.removeFirst()+" "+
        dl.size()+"\n");
    System.out.print(dl.removeLast()+" "+dl.removeFirst()+" "+
        dl.size());
}
```

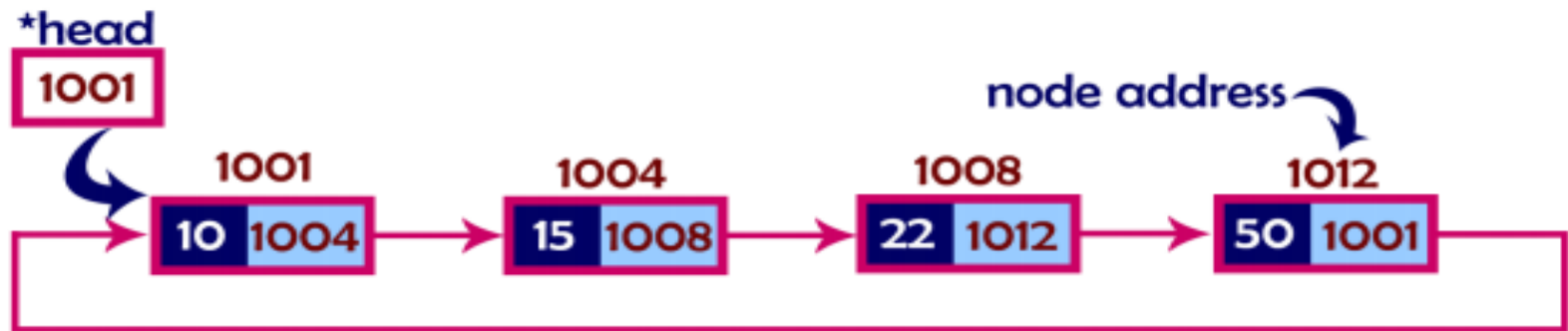
Output

0	3	2
2	1	0
null	null	0

Circularly Linked List

5

- ❑ In circular linked list, every node points to its next node in the sequence but the last node points to the first node in the list.
- ❑ That means circular linked list is similar to the single linked list except that the last node points to the first node in the list.
- ❑ Both Singly Linked List and Doubly Linked List can be made into a circular linked list



Applications of Circular Linked List

21

- ❑ Circular lists are useful in applications to repeatedly go around the list.
 - For example, many multiplayer games are turn-based, with player A taking a turn, then player B, then player C, and so on, but eventually back to player A again, and player B again, with the pattern repeating.
 - When multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then to cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application
 - Circular Linked List can also be used to create Circular Queue.

Advantages and Disadvantages of Circular Linked List

22

□ Advantages of Circular linked lists:

- Any node can be a starting point.
- Each node is accessible from any other node
- The entire list can be traversed starting from any node
- Circular lists are useful in applications to repeatedly go around the list

□ Disadvantages of Circular linked lists:

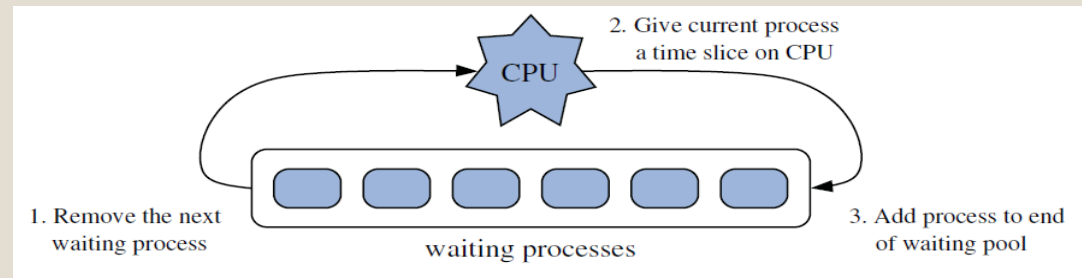
- Inserting at start of list would require doing a search for the last node.
- Finding end of list and loop control is harder (no NULL's to mark beginning and end)

Round-Robin Scheduling

23

- ❑ One of the most important roles of an operating system is in managing the many processes that are currently active on a computer, including the scheduling of those processes on one or more central processing units (CPUs).
- ❑ round-robin scheduling. A process is given a short turn to execute, known as a time slice, but it is interrupted when the slice ends, even if its job is not yet complete.
- ❑ A round-robin scheduler could be implemented with a traditional linked list, by repeatedly performing the following steps on linked list:

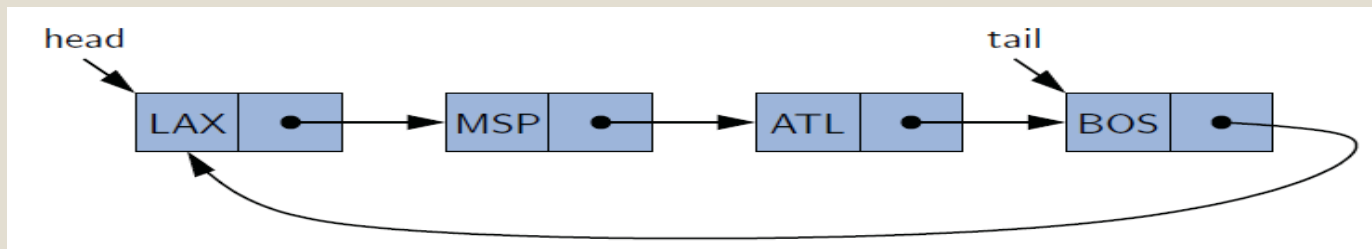
1. process $p = L.removeFirst()$
2. Give a time slice to process p
3. $L.addLast(p)$



Designing and Implementing a Circularly Linked List

24

- ❑ Circularly linked list is essentially a singularly linked list in which the next reference of the tail node is set to refer back to the head of the list (rather than null).
- ❑ Circularly Linked List class, which supports all of the public behaviors of our Singly Linked List class and one additional update method:
rotate(): Moves the first element to the end of the list.
- ❑ With this new operation, round-robin scheduling can be efficiently implemented by repeatedly performing the following steps on a circularly linked list C:
 1. Give a time slice to process C.first()
 2. C.rotate()



Basic Operations

25

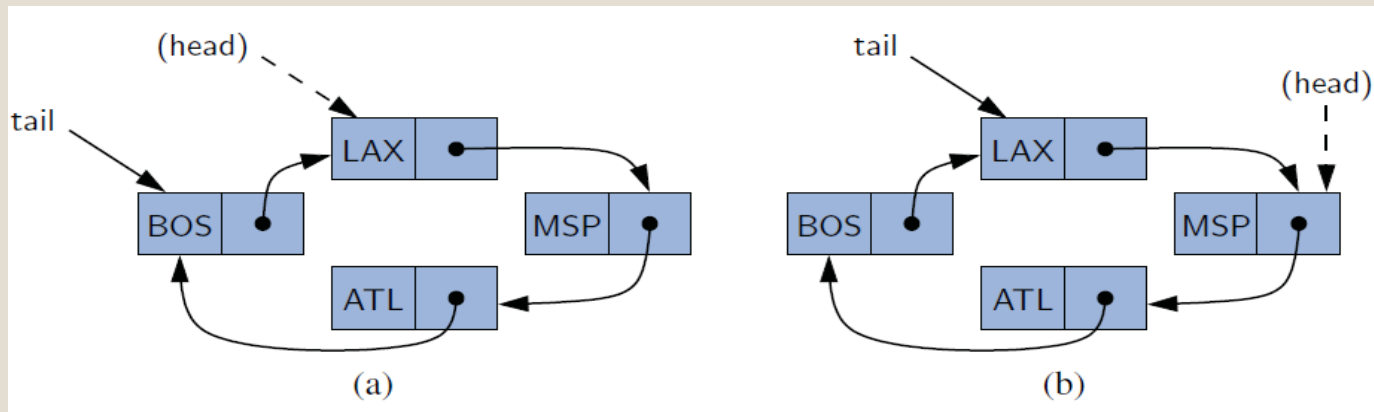
❑ Following are the basic operations supported by a Circularly Linked list.

- **Insertion** : A new node can be inserted (at the beginning, in the middle, and at the end) of the list.
- **Deletion**: It can be performed to remove node (at the beginning, in the middle, and at the tail (end) node) of the list.
- **Display**: Displays the complete list.
- **rotate()**: Moves the first element to the end of the list.
- **Search**: Searches an element using the given key.

Operations on a Circularly Linked List

26

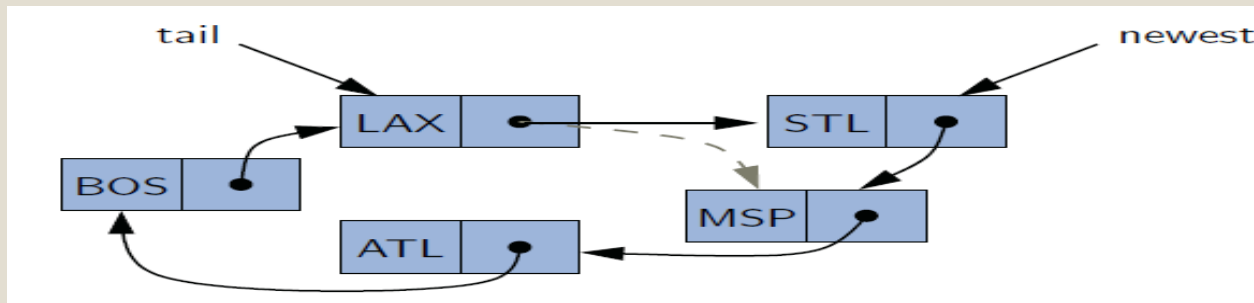
- ❑ No longer explicitly maintain the head reference.
- ❑ We can locate the head as **tail.getNext()**.
- ❑ Implementing the new rotate method is quite trivial. We do not move any nodes or elements, we simply advance the tail reference to point to the node that follows it.



Operations on a Circularly Linked List

27

- ❑ We can add a new element at the front of the list by creating a new node and linking it just after the tail of the list.
- ❑ To implement the add Last method, we can rely on the use of a call to addFirst and then immediately advance the tail reference so that the newest node becomes the last.
- ❑ Removing the first node from a circularly linked list can be accomplished by simply updating the next field of the tail node to bypass the implicit head.



Implementation

28

```
public class CircularlyLinkedList<E> {  
    //----- nested Node class -----  
    private static class Node<E>  
    {  
        private E element;           // reference to the element stored at this  
        node  
        private Node<E> next;       // reference to the subsequent node in the  
        list  
        public Node(E e, Node<E> n)  
        {  
            element=e;  
            next=n;  
        }  
        public E getElement(){return element;}  
        public Node<E> getNext() {return next;}  
        public void setNext(Node<E> n) {next=n;}  
    }  
    //----- end of nested Node class -----  
    //CircularlyLinkedList Class is Continued
```

Implementation

29

```
private Node<E> tail = null; // we store tail (but not head)
private int size = 0; // number of nodes in the list
public CircularlyLinkedList( ) { } // constructs an initially empty list
// access methods
public int size( ) { return size; }
public boolean isEmpty( ) { return size == 0; }
public E first() // returns (but does not remove) the first element
{
    if (isEmpty( )) return null;
    return tail.getNext( ).getElement( ); // the head is *after* the tail
}
public E last() // returns (but does not remove) the last element
{
    if (isEmpty( )) return null;
    return tail.getElement( );
}
//CircularlyLinkedList Class is Continued
```

Implementation

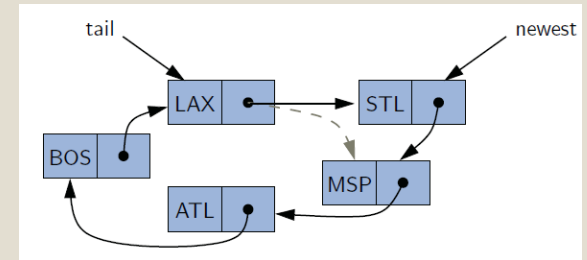
30

```
// update methods
// rotate the first element to the back of the list
public void rotate( ) {
    if (tail != null) // if empty, do nothing
        tail = tail.getNext( ); // the old head becomes the new tail
}

public void addFirst(E e) { // adds element e to the front of the list
    if (size == 0)
        {tail = new Node<>(e, null); tail.setNext(tail); } // link to
    itself circularly
    else { Node<E> newest = new Node<>(e, tail.getNext( ));
        tail.setNext(newest);
        size++;
    }
}

public void addLast(E e) { // adds element e to the end of the list
    addFirst(e); // insert new element at front of list
    tail = tail.getNext( ); // now new element becomes the tail
}

//CircularlyLinkedList Class is Continued
```



Implementation

31

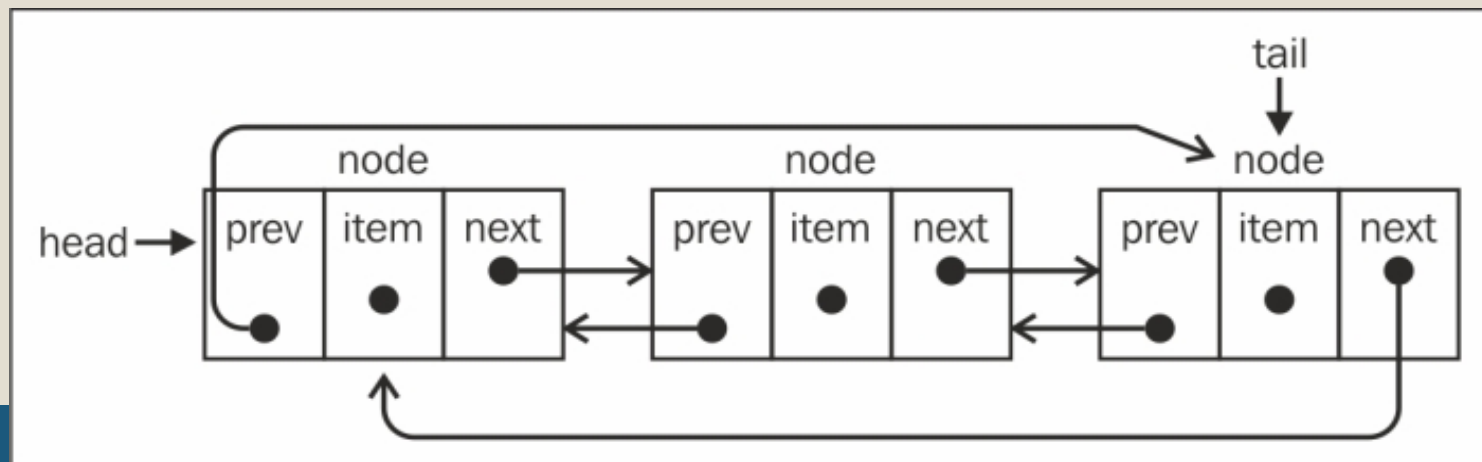
```
public E removeFirst( ) { // removes and returns the first element
    if (isEmpty( )) return null; // nothing to remove
    Node<E> head = tail.getNext( );
    if (head == tail)
        tail = null; // must be the only node left
    else tail.setNext(head.getNext( )); // removes "head" from the list
    size--;
    return head.getElement( );
}
```

Circular Doubly Linked List

32

Doubly Circular linked list has both the properties of doubly linked list and circular linked list.

- ❑ Two consecutive elements are linked by previous and next pointer . The last node points to first node by next pointer and also the previous pointer of the head node points to the tail node.
- ❑ Node traversal from any direction is possible and also jumping from head to tail or from tail to head is only one operation: head pointer previous is tail and also tail pointer next is head.



Circular Doubly Linked List

33

❑ Advantages

- Bidirectional Traversal
- No Null Pointers
- Easy Implementation of Advanced Data Structures
- Being a circular linked list tail can be reached with one operation from head node

❑ Disadvantages

- It takes slightly extra memory in each node to accommodate previous pointer
- Link pointers need to be closely monitored; otherwise, traversal can lead to infinite loop

❑ Practical Applications

- Managing songs playlist in media player applications
- Navigation Systems (forward/backward routes).
- Game Development (cyclic enemy pathing).
- Multiprocessor Scheduling (task rotation).

Basic Operations

34

- ❑ Following are the basic operations supported by a Doubly Linked List.
 - **Insertion:** Adds a new Node at the beginning and at the end of the list.
 - **Deletion :** Deletes a node at the beginning and at the end of the list.
 - **Display forward :** Displays the complete list in a forward manner.
 - **Display backward :** Displays the complete list in a backward manner.
 - **rotate():** Moves the first element to the end of the list .
 - **Search:** Searches an element using the given key.

Implementing a circular Doubly Linked List Class

35

- ❖ **size()**: Returns the number of elements in the list.
- ❖ **isEmpty()**: Returns true if the list is empty, and false otherwise.
- ❖ **first()**: Returns (but does not remove) the first element in the list.
- ❖ **last()**: Returns (but does not remove) the last element in the list.
- ❖ **rotate()**: Moves the first element to the end of the list and moves the last element to the first of the list.
- ❖ **addFirst(e)**: Adds a new element to the front of the list.
- ❖ **addLast(e)**: Adds a new element to the end of the list.
- ❖ **removeFirst()**: Removes and returns the first element of the list.
- ❖ **removeLast()**: Removes and returns the last element of the list

The Node implementation

36

```
public class CircularlyDuoblyLinkedList<E> {  
    //----- nested Node class -----  
    private static class Node<E>  
    {  
        private E element;  
        private Node<E> prev;  
        private Node<E> next;  
        public Node(E e, Node<E> p, Node<E> n)  
            {element=e; prev=p; next=n;}  
        public E getElement(){return element;}  
        public Node<E> getNext() {return next;}  
        public Node<E> getPrev() {return prev;}  
        public void setNext(Node<E> n) {next=n;}  
        public void setPrev(Node<E> p) {prev=p;}  
    } //---- end of nested Node class -----  
}
```

Implementation

37

```
private Node<E> tail=null;
private int size=0;
public CircularlyDuoblyLinkedList();
public int size(){return size;}
public boolean isEmpty(){return size==0;}
public E first()
{if(isEmpty()) return null;
return tail.getNext().getElement();
}
public E last()
{if(isEmpty()) return null;
return tail.getElement();
}
```

Implementation

38

```
public void addFirst(E e)
{
    if (tail==null)
    { tail= new Node<E>(e, null,null);
      tail.setNext(tail);
      tail.setPrev(tail);
    }else
    {
        Node<E> newest = new Node<E>(e, tail,tail.getNext());
        tail.getNext().setPrev(newest);
        tail.setNext(newest);
    }
    size++;
}
```

Implementation

39

```
public void addLast(E e)
{
    addFirst(e);
    rotate();
}
public void rotate()
{
    if (tail!=null)
    {tail=tail.getNext();}
}
public void rotateb()
{
    if (tail!=null)
    {tail=tail.getPrev();}
}
```

Implementation

40

```
public E removeFirst( )
{
    if (isEmpty( )) return null;
    Node<E> head=tail.getNext();
    E removed =head.getElement();
    tail.setNext(head.getNext());
    head.getNext().setPrev(tail);
    size--;
    return removed;
}
```

```
public E removeLast( )
{
    if (isEmpty( )) return null;
    Node<E> head=tail.getNext();
    E removed =tail.getElement();
    tail.getPrev().setNext(head);
    head.setPrev(tail.getPrev());
    tail=tail.getPrev();
    size--;
    return removed;
}
```


Any Questions?



Homework

42

□ You need to write:

- A java code to search a specific element on the list?
- A java code to insert a new node at the middle on the list after the specific element?
- A java code to remove a Node from the list using the key?

Homework

43

Circular Singly Linked List

❑ You need to write:

- A java code to search for a specific element on the list?
- A java code to insert a new element at the middle on the list after the specific element by using search?
- A java code to remove an element at the last on the list?
- A java code to remove an element at the middle on the list?

Homework

44

Circular doubly Linked List

❑ You need to write:

- A java code to search a specific element on the list?
- A java code to insert a new node at the middle on the list after the specific element?
- A java code to remove a Node from the list using the key?